

Ex. 1.0 — Three Papers on OS History and Architecture

The three papers below were located by searching the literature (Google Scholar and general web search for work on operating system history and architecture), downloaded in full and read; none of them is taken from the supplied paper list. They were chosen as a set because they form a single argument across forty-four years. Each gives a different answer to the same two questions: *where should operating system function live*, and *what may the system assume about the machine underneath it*? Multics answers with one comprehensive system on a symmetric shared-memory machine; Mach answers by decomposing that system into a minimal kernel plus user-level servers; the Multikernel answers by discarding shared memory as the organising assumption altogether. Read in order, they also show a steady rise in evidentiary standards — from a design prospectus with no measurements at all, to a status report resting on a single microbenchmark, to a broad evaluation that volunteers its own weaknesses — without any matching fall in the confidence of the architectural claim being made.

Paper 1 — Corbató and Vyssotsky, “Introduction and Overview of the Multics System” (1965)

Summary. The paper opens the six-paper Multics session at the 1965 Fall Joint Computer Conference and states the objectives the other five then elaborate. Its problem is access: batch processing had, in the authors’ words, caused user access to large machines to “retrogress,” isolating the programmer from cause and effect. Their goal is a *computer utility* — a system running continuously and reliably twenty-four hours a day like a telephone or power network, serving everything from interactive terminals to absentee (batch) jobs. The mechanisms proposed are largely hardware–software co-design on the GE 645, a modification of the GE 635. Addressing is two-dimensional: programs are written as *segments*, each of which may grow or shrink during execution (up to roughly a quarter of a million segments per user, each of up to a quarter of a million words), and segments are independently *paged* by the supervisor with a choice of 64- or 1024-word pages. Cross-segment references are bound by *dynamic linking* — a segment names another only symbolically, binding occurs automatically on first execution, and afterwards runs at full speed. Pure, recursive, shareable procedures are the normal mode of programming, so that one copy of a procedure serves many users and, as the authors note, there is “no clear-cut demarcation”

between user and system programs. Protection comes from descriptor bits acting as hardware “fire-walls,” including an execute-only bit, and most supervisor modules run with the same descriptors as user code and no access to privileged instructions. Processors form an anonymous pool with no master — “the supervisor does not have a special processor” — and reach I/O only through mailboxes in the memory modules. A file system names everything symbolically rather than by address, interlocks writing, and provides system-run backup against three named contingencies. I/O is device-independent, batch is simply the zero-terminal case of an ordinary process, resource budgets are hierarchical and delegable, PL/I is adopted for machine independence, and the reference manual is kept on-line. There is no evaluation: the system did not yet exist. The sole quantitative claim is that “a few hundred” simultaneous users are expected, derived by “simple scaling of processor and memory speed” and immediately qualified as “hazardous.” The paper closes by calling the plans “not unattainable” while conceding it would be “presumptuous” to think the initial system could meet all the stated requirements.

Critical judgment. This is a prospectus, and it cannot be judged as an empirical paper: every verb is in the future tense and nothing in it is falsifiable by its own evidence. The deeper omission is not measurement but a *cost* estimate. The paper proposes to meet “almost all of the present and near-future requirements” of a large installation and never once asks what such generality would cost to build. That is precisely where the project failed — development ran years late, Bell Labs withdrew in 1969, and UNIX was written shortly afterwards as a deliberate reaction against exactly this scale. The most accurate sentences in the paper are its own hedges. Elsewhere it argues rhetorically rather than evidentially: “machine-aided cognition,” the promised social consequences, and above all the claim that PL/I would deliver machine independence are asserted rather than demonstrated, and the last of these badly underestimated the cost of building the compiler. On substance, history has split the paper cleanly in two. Vindicated: paged virtual memory, device-independent I/O, a symbolically named hierarchical file system with per-file access control and system-provided backup, symmetric multiprocessing with no master processor, writing an operating system in a high-level language, and graded hardware protection — the descriptor bits became Multics’ rings, which x86 inherited. Refuted: user-visible *segmentation* lost decisively to the flat, paged address space, and x86-64 all but abolished segmentation; nobody today programs a two-dimensional address space, and although dynamic linking survived it did so as lazy load-time binding over a flat space, not as segment binding. The most inter-

esting case is the computer utility itself. The economic argument — that bulk economies, floor space and management efficiency favour one maximal centralised machine — was falsified for roughly thirty years by the mini-computer and the personal computer, then vindicated by cloud computing. The paper was right about the destination and wrong about both the timing and the reason, since what eventually made centralisation win was statistical multiplexing at datacentre scale, not the considerations it actually advanced.

Paper 2 — Accetta et al., “Mach: A New Kernel Foundation for UNIX Development” (1986)

Summary. Mach’s target is the accretion of mechanism in UNIX. The authors observe that a system which began with a small uniform set of operations on file descriptors had grown “a staggering number” of overlapping facilities — System V streams, 4.2BSD sockets, ptys, several kinds of semaphore, shared memory, and a “mind-boggling array” of `ioctl` operations — giving non-uniform access to resources both within a machine and across a network, while the process abstraction remained too heavyweight to express parallelism. Mach replaces the base with four abstractions. A *task* is the unit of resource allocation, holding a paged address space and a set of port rights; a *thread* is the unit of CPU utilisation, so that a UNIX process is simply a task with one thread; a *port* is a kernel-protected message queue acting as a capability, with any number of senders but exactly one receiver; and a *message* is a typed collection of data that may carry port rights and may be as large as an entire address space. Every operation on any other object is a message to the port representing it, so that the kernel is merely one server among many — the design thesis is that “the actual system running on any particular machine is a function of its servers rather than its kernel.” The virtual memory system supports per-page inheritance (shared, copy, or none) and a current-versus-maximum protection pair in which the maximum may only ever be lowered; copy-on-write implements both `fork` and bulk message transfer, illustrated with a 24 MB message that passes through a temporary kernel map. Most consequentially, page faults may be serviced *outside* the kernel by user-level pagers, so that a memory-mapped file is simply memory whose pager is the file system, and the default pager uses ordinary file systems rather than a paging partition. The implementation separates machine-dependent from machine-independent structure behind a bare `validate/invalidate/protect` interface — the independent page size is a boot-time multiple of the dependent one, 512 bytes on the VAX — letting the same machine-independent code sit above VAX page tables and

the RT/PC's inverted page table. Matchmaker, an interface definition language, generates RPC stubs for C, Pascal and CommonLisp, and user-level network servers extend ports across machines transparently. As of April 1986 Mach was binary compatible with 4.3BSD and running on assorted VAXen, a four-processor VAX 11/784 and the IBM RT/PC.

Critical judgment. The gap between the ambition of the design and the weight of the evidence is very wide, and the paper is candid that “extensive performance comparisons with other systems have not yet been done.” In practice its entire quantitative evaluation is a single number: on a MicroVAX II, touching newly allocated memory costs under 0.7 ms per 1024 bytes against roughly 1.2 ms for 4.3BSD. That microbenchmark measures zero-fill fault cost — exactly what the new VM system was built to improve — and was chosen by the implementers; `fork` is called “substantially faster” with no figure attached at all. Nothing measures IPC latency, although ports and messages are the centrepiece of the design; nothing measures multiprocessor scaling, although the abstract’s opening sentence announces a “multiprocessor operating system kernel”; threads, the feature the paper leads with, were not yet implemented and were merely “expected by Summer 1986”; and the caption of the paper’s own final figure concedes that UNIX compatibility still executed in kernel state. The central claim — that operating system services can be moved out to user-level servers at acceptable cost — is therefore precisely the claim the paper does not test. History then split the verdict. The mechanisms won comprehensively: separating the address space from the schedulable entity is now universal, memory-mapped files and external pagers are standard equipment, the machine-dependent/independent VM split survives as the `pmap` layer in the BSDs, and port-like capability handles reappear in Windows object handles and in Fuchsia’s channels, while macOS and iOS run on a Mach derivative to this day. The performance thesis lost: Mach 3.0’s user-space UNIX server proved slow, Chen and Bershad later traced the degradation to memory-system behaviour rather than to message path length, and Liedtke’s L4 work argued the cost was an artefact of implementation rather than of microkernels, cutting IPC by roughly an order of magnitude. Mach became the cautionary tale that motivated second-generation microkernels. Two of its assumptions have since expired outright. Network-transparent IPC treats a local multiprocessor and a LAN as points on one continuum, an equivalence the distributed systems community abandoned because transparency conceals latency and partial failure. And copy-on-write as the universal bulk-transfer mechanism assumed page-table manipulation was cheap relative to copying, which inverted once TLB shutdown across many cores

entered the price. Finally, the extensibility argument is made by *enumerating* UNIX’s accumulated mess rather than by showing that the port model resists the same accretion — and it did not: Mach’s own VM, with its chains of shadow objects, became notoriously intricate, and the kernel that carries its name today is large.

Paper 3 — Baumann et al., “The Multikernel: A New OS Architecture for Scalable Multicore Systems” (2009)

Summary. The Multikernel’s premise is that hardware now diversifies faster than system software can be retuned for it, so that an OS built as a shared-memory kernel with lock-protected data structures faces an unbounded engineering bill. The authors evidence this with the “heroic” removal of the Windows 7 dispatcher lock, which touched 6000 lines across 58 files, and with the many iterations required by Linux’s read-copy-update. They then argue that the machine has itself become a network — interconnects route and congest, cores differ in ISA and performance, and peripherals are already non-coherent — and propose three principles: make all inter-core communication explicit, make OS structure hardware-neutral, and treat state as replicated rather than shared. Their prototype, Barrelfish, factors the per-core OS instance into a privileged *CPU driver* (event-driven, single-threaded, non-preemptable, 7135 lines of C for x86-64) and a schedulable user-space *monitor*. Monitors hold the replicas and run the agreement protocols — one-phase commit for unmapping a page, two-phase commit for capability retyping — over URPC, a user-level message transport tuned to the cache-coherence protocol, in which the sender writes a cache line sequentially and the receiver polls its last word. Memory is managed by seL4-style capabilities with all page-table manipulation performed in user space, and a system knowledge base holding hardware facts in first-order logic computes topology-aware multicast trees at run time. The motivating measurement is stark: a single core updates shared state in under 30 cycles, but with 16 cores on the same data each update costs roughly 12,000 extra cycles, and one dedicated RPC server sustains twice the update rate of all 16 shared-memory threads together. The evaluation then shows a NUMA-aware multicast shutdown protocol holding at about 3.1k cycles across 32 cores where naive broadcast and unicast climb to roughly 13k; end-to-end page unmapping at 32 cores costing about 17.5k cycles against roughly 29.5k for Linux 2.6.26 and 46k for Windows Server 2008 R2; IP loopback at 2154 versus 1823 Mbit/s with 21 versus 77 data-cache misses per packet; UDP echo over gigabit Ethernet at 951.7 against 951 Mbit/s; and a static web

server at 18,697 requests per second against 8924 for `lighttpd` on Linux.

Critical judgment. This is much the strongest evaluation of the three and an unusually honest one — it states outright that “it would be wrong to draw any quantitative conclusions from our large-scale benchmarks,” concedes that the capability system was “a mistake...unnecessarily complex, and no more efficient” than the per-processor allocators already in Windows and Linux, calls its network stack “very much a placeholder,” and admits that every test platform was homogeneous x86-64, so the heterogeneity that motivates the whole argument was never actually exercised. That honesty makes the weak points easier to locate. The headline numbers are the least defensible: the $2.1\times$ web-server win pits a purpose-built user-space lwIP stack against general-purpose `lighttpd` on Linux with segmentation and checksum offload enabled, while Barrelfish’s own driver supported none of it, so what is measured is specialisation rather than architecture. The scalability case rests on global-coordination microbenchmarks, and there the gain comes largely from replacing serial IPs with a topology-aware multicast *tree* — an algorithmic change a shared-memory kernel could equally adopt — so the experiment does not isolate shared-nothing structure as the cause of the improvement. Reading the shutdown and unmap figures against each other is more revealing still: the raw messaging baseline costs about 3.1k cycles at 32 cores while the complete unmap costs about 17.5k, so more than four-fifths of the operation is Barrelfish’s own monitor LRPC, marshaling and unoptimised thread dispatch. The shape of the unmap curve makes the same point at the other end. Barrelfish does not start cheaper than the systems it is compared against — at two cores it costs about 10.3k cycles against roughly 5.2k for Linux and 5.8k for Windows — and it stays almost perfectly flat out to five cores before rising gently. What it buys is slope: from two to thirty-two cores Barrelfish grows about $1.7\times$, reaching some 17.5k cycles, where Linux grows $5.7\times$ to about 29.5k and Windows $7.9\times$ to about 46k. So the multikernel did not reduce the constant cost of the operation; it roughly *doubled* it and bought a much flatter curve in exchange. That is exactly the trade the authors themselves concede when they note that routing OS invocations through monitor RPC rather than system calls adds “a constant overhead on current hardware of several thousand cycles,” which is “constant as the number of cores increases” — a bargain that pays only once the machine is wide enough, and whose break-even against Linux does not arrive until around twelve or thirteen cores. The messaging table quietly reinforces the point, showing URPC at 450 cycles of latency against 424 for L4’s decade-older IPC, the real gains again being throughput and a much smaller cache footprint rather than latency. The hardware bet was

half right. Cache coherence did not collapse — commodity servers remained coherent far past 2009 — and mainstream kernels absorbed NUMA and core asymmetry using per-CPU data, RCU and sharded locks, settling in the middle of the paper’s own spectrum of sharing disciplines rather than at the shared-nothing extreme it advocates. Barrelfish remained a research vehicle. What survived is the diagnosis and the method: treating the machine as a network, which resurfaced in kernel-bypass and dataplane systems such as Arrakis and IX, and in the now-routine practice of designing OS state per-core first. One irony deserves noting — having adopted the distributed systems analogy, the paper declines its principal prize, stating that “the complete system is a failure unit,” so it pays the full price of messaging and replication without buying any tolerance of partial failure.

The Three Together

Placed side by side, the three papers are three attempts to answer the same question — what may the operating system take for granted about the machine? — and each is undone at a different layer. Multics assumed that generality could be centralised in one comprehensive system, and was defeated by the complexity budget it never wrote down. Mach assumed the boundary could be pushed outward so that services became user-level servers, and was defeated by the cost of crossing that boundary. The Multikernel assumed shared memory was itself the wrong primitive, and was overtaken by shared-memory hardware that went on scaling for another decade. The pattern in their evidence is just as instructive: 1965 offers a design prospectus with no measurements, 1986 a status report resting on one microbenchmark of the subsystem its authors had just optimised, and 2009 a broad evaluation that explicitly marks its own limits — yet the boldness of the architectural claim does not diminish as the evidence improves, which is a useful caution when reading any systems paper. Most striking is that all three have been vindicated in mechanism while being refuted in scope. Multics’ paging, protection rings and device independence are everywhere, but its segmented address space is gone. Mach’s tasks, threads, ports and memory objects are everywhere, but its argument that services could be evicted from the kernel for free is not. The multikernel’s per-core replication now sits inside Linux, but as an optimisation within a shared-memory kernel rather than as a replacement for one. The durable contribution of an architecture paper, on this evidence, is rarely the architecture.

References

- [1] F. J. Corbató and V. A. Vyssotsky. “Introduction and Overview of the Multics System.” In *Proceedings of the AFIPS Fall Joint Computer Conference (FJCC)*, vol. 27, part 1, pp. 185–196, 1965.
- [2] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevastian and M. Young. “Mach: A New Kernel Foundation for UNIX Development.” In *Proceedings of the USENIX Summer Conference*, pp. 93–112, 1986.
- [3] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach and A. Singhanian. “The Multikernel: A New OS Architecture for Scalable Multicore Systems.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 29–44, 2009.